fortranBridge: Architectural Principles, Memory Semantics, and Verification for Fortran-to-C++ and Fortran-to-Python Translation

A Technical and Algorithmic Reference Manual

scienceStack & Antigravity Scientific Team
`https://github.com/seoym3919/fortranBridge_antigravity`

October 22, 2026

Abstract

Decades of foundational scientific simulations in astrophysics, fluid dynamics, climate modeling, and particle physics exist exclusively in legacy Fortran (FORTRAN 66/77 and early Fortran 90). Migrating this critical software into modern computational ecosystems—specifically modern C++ (C++17/20) and vectorized scientific Python (NumPy, SciPy, Numba)—is hindered by profound semantic divergences: 1-based versus 0-based array indexing, column-major versus row-major memory stride order, unthread-safe `COMMON` block global state, unstructured `GOTO` control flow, and subtle intrinsic function roundoffs. This manual presents the design, algorithms, and theoretical guarantees of **fortranBridge**, an automated translation engine designed for Google Antigravity. We formulate formal transformation rules for index normalization, memory contiguity loop transposition, `COMMON` block encapsulation, formatted I/O decompilation, dataflow intent inference, multi-file topological dependency linking, and closed-loop floating-point round-trip verification certifying numerical equivalence ($\text{Rel}\, L_2 \leq 10^{-12}$).

Contents

1 Introduction and the Fortran Migration Mandate **2**

2 Indexing Paradigms and Off-by-One Guarding **2**
2.1 1-Based to 0-Based Coordinate Translation 2
2.2 Arbitrary and Negative Lower Bounds 3

3 Memory Layout and Memory Contiguity Transposition **3**
3.1 Column-Major vs. Row-Major Storage 3

4 Global State Encapsulation: COMMON and EQUIVALENCE **4**
4.1 COMMON Block State Encapsulation 4
4.2 EQUIVALENCE Statement De-Aliasing 4

5 Control Flow De-Spaghettification and Formatted I/O **5**
5.1 Arithmetic IF and Computed GOTO 5
5.2 Fortran FORMAT to C++20 std::format and Python f-Strings 5

6 Static Dataflow Intent Inference **5**

7 Multi-File Project Linking and Topological DAG **5**

8 Closed-Loop Numerical Equivalence Verification **6**

9 Summary and Command Reference **6**

1 Introduction and the Fortran Migration Mandate

Fortran has served as the backbone of high-performance scientific computing since its introduction by John Backus in 1957. Production codes built over seven decades contain millions of lines of rigorously verified numerical algorithms. However, modern scientific software infrastructure demands features absent in legacy Fortran:

1. **Object-Oriented and Functional Modularity**: Safe state encapsulation, thread safety, and dependency injection.
2. **GPU and Heterogeneous Acceleration**: CUDA, SYCL, Kokkos, and JAX/PyTorch tensor primitives.
3. **Modern Memory Semantics**: Zero-overhead abstractions like `std::span`, `std::vector`, and contiguous NumPy arrays.
4. **Interoperability and Ecosystem Integration**: REST APIs, interactive Jupyter environments, and machine learning libraries.

Manual translation of legacy Fortran into C++ or Python is notoriously error-prone. Even experienced computational scientists frequently introduce subtle off-by-one subscript errors, inadvertent cache-thrashing memory strides, and uninitialized aliasing defects. **fortranBridge** provides an automated, mathematically rigorous translation pipeline designed to solve these systemic issues.

2 Indexing Paradigms and Off-by-One Guarding

2.1 1-Based to 0-Based Coordinate Translation

The most prevalent source of catastrophic translation failure is the disparity between Fortran's default 1-based indexing and C++/Python's 0-based indexing:

$$\text{Fortran: } A(1), A(2), \ldots, A(N) \quad \longleftrightarrow \quad \text{C++/Python: } a[0], a[1], \ldots, a[N-1]. \tag{1}$$

When translating loops of the canonical form:

```
      DO 10 I = 1, N
         A(I) = B(I) + C(I)
 10   CONTINUE
```

`fortranBridge` applies unified index coordinate transformation:

$$i_{\text{C++}} = i_{\text{Fortran}} - 1, \quad i_{\text{C++}} \in [0, N-1]. \tag{2}$$

In C++, this yields:

```
for (int i = 0; i < n; ++i) {
    a[i] = b[i] + c[i];
}
```

And in Python, when loop vectorization is not possible:

```
for i in range(n):
    a[i] = b[i] + c[i]
```

Where vectorization is applicable, the entire loop reduces to a single vectorized statement:

```
a = b + c
```

2.2 Arbitrary and Negative Lower Bounds

Fortran permits arbitrary array dimensions:

$$\texttt{DIMENSION A(L_1:U_1, L_2:U_2).} \tag{3}$$

For an array declared as `DIMENSION A(-5:5, 0:10)`, the index mapping for element $A(i, j)$ into 0-based C++ coordinates is:

$$i_{\text{C++}} = i - L_1 = i - (-5) = i + 5, \quad j_{\text{C++}} = j - L_2 = j - 0 = j. \tag{4}$$

The `indexing-auditor` agent formally verifies that all shifted array subscripts are non-negative and strictly bounded by the allocated dimension extent $U_k - L_k + 1$.

3 Memory Layout and Memory Contiguity Transposition

3.1 Column-Major vs. Row-Major Storage

Consider a 2D matrix $A \in \mathbb{R}^{M \times N}$. The linear memory offset of element $A(i, j)$ differs fundamentally across programming paradigms:

$$\text{Fortran (Column-Major):} \quad \text{offset}(i, j) = (i - 1) + (j - 1) \cdot M, \tag{5}$$

$$\text{C / C++ (Row-Major):} \quad \text{offset}(i, j) = i \cdot N + j. \tag{6}$$

In Fortran, the first index i varies fastest in linear memory. Consequently, canonical Fortran algorithms nest loops with I in the innermost position:

```
      DO 20 J = 1, M
         DO 10 I = 1, N
            A(I, J) = B(I, J) * C(I, J)
 10      CONTINUE
 20   CONTINUE
```

Proposition 3.1 (Cache Line Contiguity Principle)**.** If a Fortran nested loop is translated naively to C++ without loop or array transposition:

```
// Cache-Destructive Naive Translation (Stride = N doubles per
   iteration):
for (int j = 0; j < m; ++j) {
    for (int i = 0; i < n; ++i) {
        a[i][j] = b[i][j] * c[i][j]; // Cache miss on every memory
           fetch!
    }
}
```

Memory access jumps across rows by stride N, loading a 64-byte cache line but consuming only a single 8-byte double before jumping, destroying L1/L2 data cache hit rates.

`fortranBridge` addresses this by applying **Loop Stride Inversion**:

```
// Cache-Optimal Transposed C++ Translation:
for (int i = 0; i < n; ++i) {
    for (int j = 0; j < m; ++j) {
        a[i][j] = b[i][j] * c[i][j]; // Sequential, SIMD-vectorizable
           memory loads
    }
}
```

Or, when interfacing with external Fortran-ordered libraries, explicitly retaining column-major ordering via `Eigen::Matrix<double, Dynamic, Dynamic, ColMajor>` or NumPy's `order='F'`.

4 Global State Encapsulation: COMMON and EQUIVALENCE

4.1 COMMON Block State Encapsulation

Legacy Fortran codes share variables between subroutines using global memory layouts known as `COMMON` blocks:

```
      SUBROUTINE STEP_ORBIT
      COMMON /ORBIT_DATA/ ECC, SEMI, INC, POS(3), VEL(3)
      ...
      END
```

In modern concurrent C++ and multi-threaded Python, global mutable state leads to catastrophic data races and prevents re-entrant simulation instances.

`fortranBridge` translates each named `COMMON /NAME/` block into a dedicated state structure:

```
// Modern C++ Encapsulated Struct
struct OrbitDataBlock {
    double ecc{0.0};
    double semi{0.0};
    double inc{0.0};
    std::array<double, 3> pos{};
    std::array<double, 3> vel{};
};

// Subroutine accepts state reference
void step_orbit(OrbitDataBlock& orbit_state);
```

In Python, `fortranBridge` translates the block into a typed `@dataclass`:

```
from dataclasses import dataclass, field
import numpy as np

@dataclass
class OrbitDataBlock:
    ecc: float = 0.0
    semi: float = 0.0
    inc: float = 0.0
    pos: np.ndarray = field(default_factory=lambda: np.zeros(3))
    vel: np.ndarray = field(default_factory=lambda: np.zeros(3))
```

4.2 EQUIVALENCE Statement De-Aliasing

In FORTRAN 77, `EQUIVALENCE` statements force variables to share physical memory locations:

```
      DIMENSION VEC(3)
      EQUIVALENCE (VEC(1), X), (VEC(2), Y), (VEC(3), Z)
```

The `equivalence_analyzer` engine identifies whether the aliasing represents:

- **Scalar-to-Vector Overlay**: Decompiled to C++ references (`double& x = vec[0];`) or Python tuple unpacking.
- **1D Flat Buffer to 2D Matrix Reshaping**: Decompiled to zero-copy `std::span<double>` or NumPy strided memory views (`mat = flat.view().reshape((M, N), order='F')`).
- **Type Punning**: Decompiled to C++20 `std::bit_cast` or explicit unions.

5 Control Flow De-Spaghettification and Formatted I/O

5.1 Arithmetic IF and Computed GOTO

$$\texttt{IF (E) L1, L2, L3} \iff \begin{cases} \text{Jump to } L_1 & \text{if } E < 0, \\ \text{Jump to } L_2 & \text{if } E = 0, \\ \text{Jump to } L_3 & \text{if } E > 0. \end{cases} \tag{7}$$

This is decompiled into standard C++ `if (e < 0) ... else if (e == 0) ... else ...` blocks. Similarly, `GOTO (L1, L2, L3) I` is decompiled into a C++ `switch(i)` or Python `match/case` construct.

5.2 Fortran FORMAT to C++20 std::format and Python f-Strings

Fortran formatted I/O uses cryptic descriptor strings:

```
100   FORMAT(1X, 'ENERGY = ', 1P, E15.7, 2X, 'STEP = ', I6)
      WRITE(6, 100) ENERGY, ISTEP
```

The `format_decompiler` engine maps descriptors to modern typesafe string formatters:

C++20: `std::cout << std::format(" ENERGY = {:15.7e} STEP = {:6d}\n", energy, istep);` (8)

Python: `print(f" ENERGY = {energy:15.7e} STEP = {istep:6d}")` (9)

6 Static Dataflow Intent Inference

Because FORTRAN 77 lacked argument intent keywords, all arguments were passed as mutable pointers. The `intent_analyzer` engine constructs a definition-use (def-use) dataflow graph for each routine:

- **INTENT(IN)**: Variables only read in expressions $\longrightarrow$ `const T&` or pass-by-value in C++; standard input parameter in Python.
- **INTENT(OUT)**: Variables only written to $\longrightarrow$ `T&` in C++; extracted from caller arguments and returned as clean tuple in Python (`return out1, out2`).
- **INTENT(INOUT)**: Variables both read and updated $\longrightarrow$ mutable `T&` in C++; in-place array modification in Python.

7 Multi-File Project Linking and Topological DAG

Production scientific simulations consist of numerous interdependent files. The `project_linker` engine scans codebases, identifying:

$$\text{Dependency Edges: } \mathcal{E} = \{(F_i, F_j) \mid F_i \text{ uses MODULE or calls routine defined in } F_j\}. \tag{10}$$

It executes a topological sort on the resulting Directed Acyclic Graph (DAG) to determine the strict compilation order and emits modular C++ headers (`.hpp`) with `#pragma once` alongside a turnkey `CMakeLists.txt`.

8 Closed-Loop Numerical Equivalence Verification

The `translation_verifier` and `f2py_harness` modules conduct closed-loop numerical testing comparing legacy Fortran output against translated C++ or Python implementations across parameter phase space:

$$\text{Rel}\, L_2 = \frac{\|\mathbf{y}_{\text{Fortran}} - \mathbf{y}_{\text{Port}}\|_2}{\|\mathbf{y}_{\text{Fortran}}\|_2 + \varepsilon} \leq 10^{-12}, \tag{11}$$

$$\text{Rel}\, L_\infty = \frac{\max |\mathbf{y}_{\text{Fortran}} - \mathbf{y}_{\text{Port}}|}{\max |\mathbf{y}_{\text{Fortran}}| + \varepsilon} \leq 10^{-12}. \tag{12}$$

Any discrepancy exceeding machine roundoff (10^{-15}) indicates an off-by-one indexing error, inverted coordinate frame, or precision mismatch.

9 Summary and Command Reference

Listing 1: Running fortranBridge CLI Tools

```
# Translate Fortran source file to C++ with format decompiler and
    intent inference
python3 scripts/fortran_bridge_cli.py solver.f --target cpp --out
    solver.cpp

# Translate Fortran source file to Python with an audit report
python3 scripts/fortran_bridge_cli.py solver.f --target python --out
    solver.py --report audit.md

# Scan project directory, build DAG, and generate CMakeLists.txt
python3 scripts/project_linker.py /path/to/fortran_project

# Run closed-loop numerical equivalence test suite
python3 scripts/translation_verifier.py --self-test
python3 scripts/f2py_harness.py --self-test
```